

Critical Analysis

Comprehensive Roadmap for Building a Python Machine Learning Quantitative Hedge Fund Investment Strategy

Findings	11 (2 critical, 4 high, 4 medium, 1 low)
Source document	223 lines, 7 phases
Most serious gap	No correction for multiple testing
Headline measurement	The example model adds 0.000013 of R-squared over doing nothing
Companion implementation	quantlab - 123 tests, 94% coverage
Generated	2026-09-04

Every finding names the module that addresses it. Corrections that can be enforced mechanically are enforced by tests rather than left to reviewer discipline.

Source document: [docs/source/Quantitative_Trading_Strategy_Roadmap.md](#) (223 lines, 7 phases)
Reviewed against: this repository's implementation, plus two reference repositories of working quant code.

Summary

The roadmap is a competent survey with a sound phase ordering, and it is above average for its genre in two specific respects: it names look-ahead bias and transaction costs explicitly, and its pitfalls table is genuinely good. Most introductory guides omit both.

It also contains one worked code example that does not work, one piece of cross-validation advice that is self-contradictory and would silently invalidate any result built on it, and several omissions that matter more than anything it covers. The most serious omission — no correction for multiple testing — is the single thing most likely to separate a strategy that makes money from one that only looked like it would.

The findings below are ordered by how much damage each does if followed as written. Each names the module in this repository that addresses it.

#	Finding	Severity	Addressed by
1	k-fold CV advice contradicts its own warning about look-ahead bias	Critical	validation/splits.py
2	The example model is a tautology that cannot trade	Critical	labeling/targets.py
3	Models selected on ML metrics, not economic ones	High	metrics/performance.py
4	No correction for multiple testing	High	metrics/deflated.py
5	Survivorship bias never mentioned	High	data/universe.py
6	No point-in-time discipline for fundamentals	High	data/universe.py
7	No labeling methodology	Medium	labeling/targets.py
8	Execution timing left unspecified	Medium	backtest/engine.py
9	Kelly sizing recommended without its caveat	Medium	backtest/sizing.py
10	Cost model omits borrow and market impact	Medium	backtest/costs.py
11	Dated library recommendations	Low	features/technical.py

Finding 1 — The cross-validation advice contradicts itself · Critical

Where: §4.2 "Train and Validate Models"

The roadmap says:

- **Cross-Validation:** Apply k-fold cross-validation, ensuring no look-ahead bias.
- **Action:** Use Scikit-learn's `train_test_split` and `GridSearchCV` for training and tuning.

These two lines cannot both be followed. Standard k-fold cross-validation **shuffles** the data — that is look-ahead bias, not a thing you can apply k-fold "while ensuring" the absence of. And the two tools named make it worse, because both default to exactly the wrong behaviour:

- `train_test_split` defaults to `shuffle=True`.
- `GridSearchCV` defaults to `KFold`, which for a classifier means `StratifiedKFold` — also shuffled relative to time order.

A model cross-validated this way trains on Thursday and Friday to predict Wednesday. It will report a strong out-of-sample score that cannot be reproduced in live trading, because live trading does not supply next week's prices.

Note that the roadmap gets this *right* one line earlier, under "Data Splitting" ("use time-based splits"), and then contradicts it. A reader following the Action line — the one with the code in it — gets the broken version.

Why `TimeSeriesSplit` alone is not the fix. Respecting time order is necessary but not sufficient. With a label spanning `h` bars, the last `h` training labels are computed from prices that fall inside the test window. The training set therefore contains information about the test period even though every training *timestamp* precedes it.

This is not hypothetical for this user. In the reference repository [KingZTheShadowz](#), the script `QS007-ML-in-finance/01_ml_trend_detection.py` builds its target as:

```
df['target'] = df['price_above_ma'].astype(int).shift(-5)
```

Each label spans the next 5 bars, so consecutive labels overlap by 4. That script splits on a single date with no purge, so the training rows immediately before the cut-off are labelled with prices from after it.

The fix — two defences, both implemented in `validation/splits.py`:

- **Purge:** drop training rows whose label horizon reaches into the test window.
- **Embargo:** additionally drop rows immediately *after* the test window, since serial correlation means an adjacent bar still carries information about it.

```
from quantlab.validation.splits import PurgedWalkForwardSplit

splitter = PurgedWalkForwardSplit(n_splits=5, purge=5, embargo=5, min_train=500)
```

`tests/test_splits.py::test_zero_purge_leaves_labels_overlapping` asserts the unpurged case is detected, so the guard cannot be quietly removed.

Reference: López de Prado, *Advances in Financial Machine Learning*, ch. 7.

Finding 2 — The example model is a tautology · Critical

Where: §"Example Python Code Snippet", lines 190–215

The roadmap's only complete code example is:

```
data['Return'] = data['Close'].pct_change() # computed, then never used
data['Lag1'] = data['Close'].shift(1)

X = data[['Lag1']] # yesterday's closing price
y = data['Close'] # today's closing price
```

This predicts today's close from yesterday's close. Since consecutive daily closes differ by a fraction of a percent, the fit is nearly perfect and completely empty.

Measured, not asserted. `scripts/demonstrate_tautology.py` runs this exact code and reports:

Measurement	Value	What it means
R-squared	0.985021	Looks excellent
Fitted coefficient	0.998155	The model learned $y = x$
R-squared from <i>just repeating the input</i>	0.985008	Identical to 6 s.f.
Value added by fitting the model	0.000013	Essentially nothing
Directional accuracy	0.5115	A coin flip

The regression's entire contribution is thirteen millionths of R-squared over the strategy "guess that today's price equals yesterday's." And it cannot call the direction of the next move, which is the only thing a trader needs.

There is a second, deeper problem. Both series have a unit root, so regressing one on the other is **spurious** in the Granger–Newbold sense: the high R-squared is an artefact of the shared trend, not evidence of a relationship. It would look just as impressive between two unrelated random walks.

The fix: predict **returns**, never price levels. Returns are (approximately) stationary, and their R-squared is honest about how little is predictable. The same script's corrected version reports R-squared of **-0.004** — a much worse-looking number that is far more useful, because it correctly says one lagged return barely predicts the next.

This is enforced structurally rather than by convention: `labeling/targets.py` only ever produces return-based targets, and `validation/leakage.py::assert_no_future_columns` raises on a feature that tracks the contemporaneous close too closely. On this project's data the roadmap's `Lag1` feature scores **0.998** on that check while the most price-coupled legitimate feature reaches only **0.65** — the two populations separate cleanly.

(Minor, same snippet: `data['Return']` is computed on line 198 and never referenced again — a leftover that hints the example was meant to use returns.)

Finding 3 — Model selection on ML metrics rather than economic ones · High

Where: §4.3 "Evaluate Performance"

The roadmap evaluates with MSE, R-squared, accuracy, precision, recall and F1. Every one of these is blind to the thing that determines profit: **magnitude**.

A strategy can be right 70% of the time and lose money steadily, if the 30% of occasions it is wrong are the large moves. Payoffs in markets are asymmetric, and a classification metric weights a 5bp win exactly like a 500bp loss.

Accuracy is also actively misleading on imbalanced targets. A label with a 16% base rate is 84% "accurate" from a model that never predicts the positive class at all.

The fix: select on the net-of-cost return series. `metrics/performance.py` provides Sharpe, Sortino, max drawdown, Calmar, hit rate and turnover, and the pipeline reports them from the backtest rather than from the classifier. Classification metrics are diagnostics, not objectives.

Finding 4 — No correction for multiple testing · High

Where: absent throughout; §4.2 and Phase 5 both imply extensive iteration

This is the most important omission in the document.

The roadmap describes an iterative research loop — try features, try models, tune hyperparameters, compare against a benchmark — and then reports a Sharpe ratio. But **if you try N strategies and report the best one, that Sharpe is biased upward by construction**. Even with no real edge anywhere, the maximum of N sample Sharpe ratios grows roughly as $\sqrt{2 \cdot \log N}$ times their standard error.

Concretely: try 50 variants on five years of daily data with zero true edge, and the best one will show a Sharpe near 0.9. Most people would trade that.

The roadmap's pitfalls table has an "Overfitting" row recommending cross-validation and regularisation. Those address a *model* overfitting its training data. They do nothing about a *researcher* overfitting the backtest by selection — a different failure that survives perfect cross-validation, because each individual candidate was

validated honestly and the bias enters at the moment you pick the winner.

The fix: `metrics/deflated.py` implements

- `deflated_sharpe_ratio` — the probability that an observed Sharpe reflects skill rather than the best of `n_trials` lucky draws, accounting for skew and kurtosis (both of which make a given Sharpe *less* impressive by widening its sampling error).
- `minimum_track_record_length` — how long a record must run before its Sharpe is distinguishable from noise. The answer is routinely years, which is worth knowing before committing capital on a two-year backtest.
- `probability_of_backtest_overfitting` — the CSCV procedure: over every balanced in-sample/out-of-sample recombination, how often does the in-sample winner land in the bottom half out of sample?

The pipeline prints the deflated Sharpe next to the raw one and states plainly whether the result clears the hurdle. On this repository's structureless synthetic data it reports **0.000** — correctly refusing to credit a null strategy.

References: Bailey & López de Prado, "The Deflated Sharpe Ratio" (2014); "The Probability of Backtest Overfitting" (2016).

Finding 5 — Survivorship bias is never mentioned · High

Where: §2.1, §2.2, §3.1

The roadmap's worked objective is:

"Predict daily closing prices for S&P 500 stocks to inform trading decisions."

Backtesting on *today's* S&P 500 constituents is the canonical survivorship trap. Today's membership list is the outcome of a selection you could not have made at the time: firms that collapsed, got acquired, or were delisted are silently absent. Every one of today's members survived and, by construction, was large enough to stay in the index.

The effect is not small — it is large enough to make a mediocre strategy look good, and it flatters long-only equity strategies most.

The pitfalls table covers overfitting, look-ahead bias, non-stationarity, black boxes, transaction costs and data quality. Survivorship bias belongs in that table and is not in it.

The fix: `data/universe.py` provides `PointInTimeUniverse`, which stores membership intervals so a delisted name remains tradable over the period it was actually a member. `StaticUniverse` carries an explicit `is_point_in_time = False` flag, so a caller can refuse it for an index study rather than discovering the bias in the results.

Finding 6 — No point-in-time discipline for fundamental data · High

Where: §3.1 "Gather Relevant Data"

The roadmap recommends fundamental data — "financial statements, earnings reports" — with no mention of *when that data became known*.

Two distinct problems:

- 1 **Reporting lag.** Q4 results are not available on 31 December. Using them at that date is a look-ahead of weeks.

- 2 **Restatement.** Financial statements get revised. A database queried today returns the *restated* figures, which nobody had at the original date.

Both quietly inflate results, and neither is caught by any amount of careful time-series splitting, because the timestamps look correct — it is the *values* that are from the future.

The fix: any fundamental source must be as-reported with a `known_on` date, and joins must use that date rather than the fiscal period end.

Finding 7 — No labeling methodology · Medium

Where: §2.1, §4.1

The roadmap frames the target as "predict the next day's stock price" and never revisits the choice.

A fixed-horizon label ignores the path. A position that would have been stopped out on day 2 is still scored on its day-5 return, so the label describes an outcome the strategy would never have experienced. Fixed thresholds also make the class balance swing with the volatility regime: the same "+1% in 5 days" threshold is common in a turbulent market and rare in a calm one.

The fix: `labeling/targets.py` implements

- **vol-scaled thresholds**, so the hurdle scales with volatility *knowable at entry* and the same label means the same thing across regimes;
- `triple_barrier`, which walks each path forward and labels by whichever barrier is touched first, or 0 if the horizon expires untouched.

The hurdle is an explicit, documented parameter (`threshold_vol_multiple`) rather than an accident of the code. Setting it to a full 1sigma leaves only ~17% positives — too imbalanced to learn from — which is exactly the kind of choice that should be visible in a config rather than buried in an expression.

Finding 8 — Execution timing is left unspecified · Medium

Where: §5.1, §5.2

Phase 5 covers backtesting frameworks and transaction costs but never states **when a signal gets filled**. That omission hides a large share of phantom alpha.

A signal computed from the close of bar t cannot be filled at that same close — the close has already happened by the time you observe it. Filling there is a one-bar look-ahead. It is especially destructive for mean-reversion signals, because those predict precisely the move you would be pretending to trade at.

The fix: `backtest/engine.py` makes the convention explicit and default: signal at close t -> fill at the **open of bar $t+1$** . The alternative `same_close` mode exists only so the size of the error can be measured, and `tests/test_backtest.py::test_same_close_execution_inflates_returns` asserts it produces higher returns than the honest convention — quantifying the trap rather than just warning about it.

Finding 9 — Kelly sizing recommended without its caveat · Medium

Where: §6.2 "Implement Risk Controls"

Position Sizing: *Use methods like the Kelly criterion for optimal capital allocation.*

Kelly is optimal only when the edge is **known**. Applied to an *estimated* edge — which is all anyone ever has — it is notoriously unstable, because the optimal fraction is roughly linear in a quantity whose estimation error is

large. A modest overestimate produces a large overbet, and Kelly's drawdowns are severe even when the edge is entirely real.

The fix: `backtest/sizing.py` provides `fractional_kelly` (defaulting to a quarter) and `vol_target_position`, which is the default. Vol targeting also gives the portfolio a stable risk profile across regimes, which full Kelly does not.

Finding 10 — The cost model omits borrow and market impact · Medium

Where: §5.2

The roadmap names commissions, spreads and slippage — better than most guides — and stops there. Two costs are missing:

- **Borrow / financing.** A short position pays a stock-loan fee daily. At 50bp annually on a fully-invested short book that is 0.5% of NAV per year, larger than the edge of many published strategies.
- **Market impact.** Slippage is treated as a constant, but impact grows with participation rate. The square-root law is the standard approximation, and a strategy that is profitable at \$1m can be unprofitable at \$100m for this reason alone.

The fix: `backtest/costs.py` separates turnover costs (paid when the position *changes*) from holding costs (paid every bar the position is *open*), charges borrow on the short side only, and supports square-root impact. `test_a_high_turnover_strategy_is_destroyed_by_costs` asserts that a signal flipping every bar cannot survive realistic costs.

Finding 11 — Dated library recommendations · Low

Where: §1.1

Recommended	Issue
Zipline	Unmaintained since 2020. The live fork is <code>zipline-reloaded</code> .
TA-Lib	Requires a C library built and on the linker path — a recurring install failure. <code>pandas-ta</code> , or a few lines of vectorized pandas, is a better dependency for a reproducible repo.
Alpha Vantage	Free tier is 25 requests/day, impractical for a universe of any size.

This project implements its indicators directly in `features/technical.py` — each is a handful of lines and unit-tested against known values, which removes a fragile build dependency entirely.

A related warning from the user's own code. The `KingZTheShadowz` repository is built on OpenBB SDK v3, which has been superseded by v4 with a different API. `QS010-relative-strength-index/01_rsi.py` already carries a commented-out compatibility block:

```
# # OPENBB 4 Compatibility:
# import openbb as openbb # openbb 4
# df = openbb.obb.equity.price.historical(...).to_df() # openbb 4
```

That is vendor churn leaking into 26 separate scripts. This repository puts data access behind a single `PriceProvider` protocol so the churn is confined to one file.

What the roadmap gets right

Worth stating plainly, since the above is uniformly critical:

- **The phase ordering is correct.** Learn -> define objective -> data -> model -> backtest -> risk -> deploy is the right sequence, and the insistence on defining the objective *before* modelling is advice many skip.
- **It names look-ahead bias and transaction costs explicitly.** Most introductory material omits both entirely.
- **The pitfalls table is genuinely good** — non-stationarity, black-box opacity and data quality are all real, and SHAP/LIME is the right pointer for interpretability.
- **"Start with simpler models before exploring deep learning"** is correct and frequently ignored. On daily bars with a few thousand rows and this signal-to-noise ratio, a deep network has far more capacity than the data can identify.
- **The "Realistic Expectations" section is honest** about markets being unpredictable and losses being likely. That candour is rarer than it should be.
- **Paper trading before live capital** (§7.1) is the right gate.

How the roadmap's phases map to this repository

Roadmap phase	Implemented in	Status
1 — Preparation	—	Background reading; no code
2 — Define strategy	<code>config.py</code> , <code>configs/</code>	Implemented
3.1 — Data collection	<code>data/providers.py</code> , <code>data/cache.py</code>	Implemented (synthetic/CSV; yfinance untestable here)
3.2 — Cleaning	<code>data/providers.py::validate_ohlcv</code>	Implemented
3.3 — Feature engineering	<code>features/</code>	Implemented
4.1 — Model choice	<code>models/</code>	Implemented (RF, logistic)
4.2 — Train and validate	<code>validation/</code>	Implemented with the correction from finding 1
4.3 — Evaluation	<code>metrics/</code>	Implemented on economic metrics, per finding 3
5 — Backtesting	<code>backtest/</code>	Implemented with stated execution timing, per finding 8
6 — Risk management	<code>backtest/sizing.py</code> , <code>backtest/costs.py</code>	Implemented
7.1 — Paper trading	—	Not implemented; needs a broker connection
7.2 — Live execution	—	Not implemented; needs a broker connection
7.3 — Monitoring & retraining	—	Not implemented
—	<code>metrics/deflated.py</code>	Added — has no counterpart in the roadmap (finding 4)
—	<code>data/universe.py</code>	Added — has no counterpart in the roadmap (finding 5)

The reference repositories

Repository	What it is	What it lacks
KingZTheShadowz	26 standalone scripts (QS001–QS026) demonstrating individual techniques: RSI, MACD, ATR, HRP, k-means, autoencoders, Markov models. Good breadth.	No package structure, no tests, no shared abstractions. <code>requirements.txt</code> is corrupt — UTF-16 encoded with interleaved spaces, so <code>pip install -r</code> cannot read it. Built on the superseded OpenBB v3 API.
KingZTheShadowz07	Fork of <i>awesome-systematic-trading</i> : a 578-line curated index plus 65 QuantConnect strategy implementations. Excellent as a source of strategy ideas.	A catalog, not a runnable system. The strategies target the QuantConnect platform (<code>QCAgorithm</code>) and do not run standalone.

Neither has automated tests, and neither separates research from execution. This repository is intended to be the layer they lack: the scripts remain a source of technique, and this provides the validated harness to evaluate them honestly.

Recommended reading

Ordered by relevance to the gaps above, and more specific than the roadmap's own list:

- 1 **López de Prado, *Advances in Financial Machine Learning*** (2018) — chapters 3 (labeling), 7 (purged CV) and 11–12 (backtest overfitting) address findings 1, 4 and 7 directly.
- 2 **Bailey & López de Prado, "The Deflated Sharpe Ratio"** (2014) — finding 4.
- 3 **Harvey, Liu & Zhu, "...and the Cross-Section of Expected Returns"** (2016) — why a t-statistic of 2.0 is nowhere near enough after decades of collective data mining.
- 4 **Jansen, *Machine Learning for Algorithmic Trading*** (2nd ed.) — the roadmap already lists this, and it is the best practical companion.

Bottom line

The roadmap is a reasonable map of the territory and a poor set of turn-by-turn directions. Its structure is worth keeping; its §4.2 cross-validation advice and its example code should not be followed as written, and the absence of any multiple-testing correction is what would most likely turn a promising backtest into a losing live strategy.

Each finding above is addressed by a specific module here, and the corrections that can be enforced mechanically are enforced by tests rather than left to discipline — `tests/test_leakage.py` in particular exists so that findings 1 and 2 cannot silently reappear.